Elements: CryoStack — Reusable Cyberinfrastructure for Reproducible Cryosphere Computing

1 Intellectual merit

Ice-sheet science increasingly depends on integrating nonlinear numerical models, satellite, airborne, radar, and in-situ observations, ensemble data assimilation (DA), and scalable computing. DA provides a principled means to update model state and uncertain parameters while representing forecast uncertainty [Evensen, 2003; Gillet-Chaulet, 2020; Choi et al., 2025]. In practice, however, an end-to-end DA experiment remains a systems-integration project. A scientist must install a model and its compiled dependencies, translate fields between model and DA representations, stage observations, configure MPI, submit and monitor scheduler jobs, recover outputs, and preserve enough metadata to reproduce the calculation. These tasks are repeated for every model and computing site. They divert scientific effort, discourage cross-model studies, slow collaboration, and exclude groups that lack dedicated research-software engineering or HPC support. The underlying problem is broader than any one model or algorithm: community applications repeatedly rebuild authentication, configuration, storage, visualization, provenance, and remote-execution infrastructure instead of sharing sustainable services.

The project team has established exploratory components that demonstrate feasibility but are not yet community cyberinfrastructure. ICESEE is a Python/MPI ensemble DA research framework with demonstrated ISSM and Icepack couplings [Kyanjo et al., 2026a], executed through its own Local, Remote, and Cloud implementations. CryoLauncher configures and runs ISSM and Icepack through Remote (Direct SSH or Connector/Relay to institutional Slurm/PACE resources) and Cloud (AWS Batch on Fargate or EC2 On-Demand) execution paths, with no local execution mode, and is being decomposed into reusable frontend panels, execution backends, remote drivers, and cloud-provider interfaces. A shared gateway now provides salted-password, GitHub, and ORCID authentication; schema-versioned configurations and workspaces; experiment records and event timelines; and a role-based Control Center for users, experiments, authentication, diagnostics, and audit events. LIVIST (Living Ice Sheet Temperature) explores Antarctic englacial-temperature products inferred from radar and constrained by boreholes. The new Frozen Legacies application registers historical Antarctic radar collections through dataset manifests and adapters, builds catalog and GeoJSON products, and exposes flightlines, observation points, derived LYRA products, and the ASTRA, ARIES, TERRA, and URSA interpretation tools. CryoLauncher and ICESEE implement their own submission, execution, and result-handling logic rather than a shared pipeline, but reuse the same Connector/Relay connectivity and a bring-your-own-AWS onboarding path – connected through connection-scoped CloudFormation, so independently created connections do not depend on one globally fixed execution role – instead of duplicating separate cloud stacks. On that shared onboarding path, CryoLauncher's ISSM workflow has completed end-to-end execution on both AWS Batch Fargate and EC2 On-Demand, including MPI-parallel solver execution, postprocessing, result retrieval, and visualization, and has reached the Georgia Tech institutional MATLAB license through the same Connector/Relay infrastructure used for Remote access; ICESEE's own Cloud path has separately executed its Lorenz-96 example end to end on Fargate at a single process (NP=1). Together these components establish technical feasibility for a shared data-to-model environment [Kyanjo and contributors, 2026]; demonstrated end-to-end execution does not establish a scientifically qualified or secure cloud lifecycle, and the components do not yet provide a stable community plug-in contract, production-safe HPC bridge, qualified cloud lifecycle, complete provenance, or distributed operations model.

Operational evidence from the current deployment complements this technical feasibility. The

four applications run behind one gateway that shares identity, roles, workspaces, and the Control Center while each keeps its own scientific interface and data or execution path; a dependency-aware registry builds the Frozen Legacies, LIVIST, and documentation components, and health checks cover the hosted applications. The October 2026 release added login-protected application routes, gateway-only access to the notebook servers, per-user grants for internal and relay requests, an allow-listed notebook environment that excludes gateway credentials, and a backup procedure verified by restoring production data into an isolated copy; its working tree passes 3,649 automated tests. This evidence describes a deployed and tested working tree, not yet a committed or archived source release, and it does not remove the limitations below.

CryoStack is designed as general scientific cyberinfrastructure rather than a wrapper around these initial applications. Scientific tools retain their domain-specific interfaces and capabilities while reusing identity, configuration, experiment management, execution, storage, visualization, deployment, and provenance services. This award commits to PISM as the new production model adapter and to a second Frozen Legacies dataset adapter as a test of data-side extensibility. It establishes the contracts through which future models, data collections, preprocessing utilities, and analysis tools can enter without redesigning the platform. Other models such as Elmer/Ice and SICOPOLIS remain a forward pathway, not award deliverables.

These components constitute a credible prototype, not yet sustained cyberinfrastructure. The modular migration is incomplete; application and dataset contracts are not versioned public SDKs; tests do not form a common scientific acceptance gate; connector sessions are bound to their owner but not to resource policy or a durable session store; broad remote commands and an optional password-bearing bootstrap remain; notebook kernels still run under the gateway's operating-system account, and the per-user kernel sandbox is tested but not enabled in production; production state uses SQLite, and the health checks are not an external availability monitor; the demonstrated CryoLauncher AWS workflow is not yet CryoBench-qualified for production use and EC2 Spot, GPU, and multi-node provisioning remain scientifically unvalidated; metadata and licenses disagree across packages; and adoption is not yet measured against public targets. The central premise of the proposed project is that scientific applications should be able to evolve independently while reusing secure, portable, and measurable services for identity, experiment management, execution, provenance, storage, and visualization. Achieving this requires research and engineering beyond routine operation of the prototype: generalizing interfaces, replacing unsafe remote-control patterns, qualifying science across heterogeneous resources, evaluating users, and distributing maintenance beyond one developer. The project tests the hypothesis that models and observational datasets with different interfaces, formats, and scientific uses can share one execution, provenance, and access infrastructure without losing domain-specific scientific meaning, provided that interoperability is defined by versioned, scientifically validated contracts rather than by convention, and that doing so reduces the effort required to reproduce, extend, and sustain cryosphere workflows. Three integrated objectives test this hypothesis: O1 establishes scientifically meaningful interoperability, O2 qualified and reproducible execution, and O3 reproducible use, evaluation, and sustained access:

O1 Develop and validate scientifically interoperable and extensible infrastructure for cryosphere models and observational datasets. Complete the modular platform migration; define versioned contracts and templates; deliver PISM and a second historical-radar dataset adapter; and build CryoBench qualification gates.

O2 Develop and evaluate reproducible, secure, and resource-aware scientific execution across

heterogeneous computing environments. Develop a typed FastAPI control plane and least-privilege connector/relay; replace unrestricted remote execution with allow-listed operations; implement least-privilege credential handling, per-user kernel isolation, policy-controlled computing allocations, and audit trails; package reproducible environments; and qualify consistent workflows on Georgia Tech PACE and a user- or project-managed AWS deployment.

O3 Establish and evaluate reproducible scientific experiment services and sustainable community-access mechanisms for cryosphere research. Extend the existing identity, experiment, Control Center, and deployment foundations into durable provenance and operations services; connect Frozen Legacies or LIVIST products to modeling/DA; evaluate all four applications through representative scientific tasks; and establish contribution, release, access, and governance processes.

The requested three-year Elements award is the appropriate scale: it will harden and generalize an early research prototype while focusing the new scientific integration on one high-value community model, PISM, and one new historical-radar collection. Every future application or dataset adapter will reuse the same identity, validation, execution, provenance, deployment, and operations services, and every qualified backend will serve all executable applications. This gives CryoStack high growth potential: the marginal cost of future integrations falls while the scientific and preservation value of the shared foundation grows. It will not create a new ice-sheet model, operate a general-purpose commercial cloud, or promise permanent compute to all users.

2 State of knowledge

2.1 Fragmented scientific software and the missing community layer

ISSM, Icepack, and PISM embody different numerical methods, languages, dependency stacks, restart conventions, and execution assumptions [Larour et al., 2012; Shapero et al., 2021; PISM Authors, 2026]. This diversity is scientifically valuable, but each surrounding workflow separately implements installation, configuration, state translation, observation handling, job submission, and result packaging. Ensemble DA adds deterministic member initialization, state packing and unpacking, observation operators, synchronized failure handling, parallel I/O, and diagnostics. Current tools do not provide a common path by which an independently developed model can declare these capabilities, pass scientific and operational qualification, and become portable across resources. CryoStack's new backend interfaces and frontend panels establish the code boundaries for such a path, but still wrap portions of the original monolithic implementations.

The same gap separates observations from simulation. LIVIST serves derived temperature products, while Frozen Legacies now demonstrates manifest-driven registration, ingestion adapters, catalog/GeoJSON generation, and preservation of historical radar interpretation tools. Its review workflow records the stage and version of each submitted interpretation, and automatically generated URSA picks carry source, parameter, and per-frame quality provenance while remaining marked unvalidated; LIVIST records the upstream identity of each input and compares generated products by checksum. These data applications need provenance and review semantics that a model-execution interface does not express, which is why dataset and model contracts must be distinct. Only the initial LYRA adapter is registered, however, and desktop processing is not yet a browser-executable, provenance-captured workflow. Without versioned dataset and observation-bundle contracts, units, coordinates, uncertainties, quality controls, transformations, source iden-

tifiers, and licenses can be lost during handoff. A community layer should preserve specialized interfaces while making exchanged artifacts explicit and reproducible.

2.2 Heterogeneous computing and science-gateway execution

Scientific workflows must run where allocations, data, licensed software, and suitable hardware exist. Institutional clusters often use Slurm and restrict inbound traffic; cloud systems require cost, identity, quota, and lifecycle controls. Containers and Spack improve environments but do not supply job authorization, staging, monitoring, cancellation, recovery, or a portable record of execution. The unresolved challenge is a narrow bridge between a public control plane and protected, user-authorized resources. Storing long-lived private keys or exposing arbitrary shell commands is unacceptable; requiring users to rebuild scheduler and provenance logic at each site defeats the gateway's purpose.

2.3 Reproducibility, qualification, and proof of concept

Source code alone does not reproduce an experiment. The record must bind code and adapter versions, configuration, inputs and transformations, environment digests, resource request, random seeds, scheduler identity, logs, outputs, checksums, and diagnostics. Nor does a zero exit code establish scientific validity: conservation, restart consistency, assimilation error and spread, and model-specific tolerances require explicit tests.

ICESEE, CryoLauncher, LIVIST, Frozen Legacies, authentication and experiment services, the Control Center, packaging recipes, and the deployment registry provide a proof of concept for a shared environment. They also expose the remaining gap. Interfaces are not yet stable external SDKs; shared platform tests do not yet gate scientific or cross-backend execution equivalence; the connector command channel is too broad for public use; notebook kernels are not yet isolated from the gateway's operating-system account; credential, relay, and recovery semantics have not received external review; PISM is not integrated; the CryoLauncher AWS workflow executes end to end for ISSM and Icepack but is not yet CryoBench-qualified, and ICESEE's own Cloud path remains limited to a single-process Lorenz-96 demonstration; provenance is not yet a stable cross-application schema; and adoption is not measured against public targets. The proposed award is therefore not funding an already completed portal. It funds the distinct work required to convert promising components into trusted, extensible community infrastructure.

2.4 Scientific opportunity and community impact

The central science outcome is a reproducible path from observations to multi-model estimates of ice-sheet state and uncertain parameters. PISM is an open-source, parallel, thermodynamically coupled ice-sheet model that has been used in cryosphere research for roughly two decades [PISM Authors, 2026]. Its addition gives CryoStack a complementary model architecture and a direct path to a mature user community. The project will support three motivating use cases:

- PISM state and parameter estimation. Assimilate synthetic and observational surface velocity and elevation products to estimate geometry and selected basal or rheological parameters in a regional Greenland test domain. The use case will include an observing-system simulation experiment with known truth and a documented real-data demonstration.
- Cross-model reproducibility. Run a common observation bundle, ensemble specification, and diagnostic definition through two or more supported adapters. Differences can then be attributed to model structure or configuration rather than undocumented staging and post-processing.

- Data-to-model workflow. Define a versioned observation-bundle schema and demonstrate the path from a Frozen Legacies radar-derived product or LIVIST temperature product to model initialization, validation, or an assimilation experiment. The schema will record variable definitions, coordinates, units, uncertainty, source DOI/URL, spatial/temporal coverage, and transformations.

A parallel preservation use case will register one additional historical radar collection through the Frozen Legacies manifest/adapter path, record source and transformation provenance, and publish machine-readable catalog, geospatial, quality-control, and downloadable derived products. This tests whether the same platform can grow through data contributions as well as model contributions.

The work benefits users beyond the initial PISM target because the same adapter, validation, provenance, and execution contracts apply to glacier, climate, and geophysical models. Tutorials will be usable on a laptop with an idealized model before a user requests HPC access, enabling education and participation without making training the primary project activity.

2.5 Innovation required beyond the prototypes

The innovation is not a new web skin around existing codes. CryoStack will make scientific interoperability an executable and testable contract across applications, models, datasets, and compute resources. A model adapter will expose: model identity and version; configuration schema; initialization and forecast operations; state-vector packing/unpacking; observation operators; restart/checkpoint behavior; resource estimates; and output/provenance declarations. Adapters will be isolated from execution backends, so the PISM integration will not contain PACE- or AWS-specific job logic. Conversely, execution backends will submit a typed experiment without knowing the internal details of PISM or ISSM.

This separation makes the experiment specification portable across resources. The current prototype already demonstrates this informally: the same ISSM or Icepack configuration submitted through CryoLauncher runs on institutional Slurm via Remote execution or on AWS Batch via Cloud execution without a backend-specific rewrite of the scientific application, and ICESEE separately demonstrates the same property across its own Local, Remote, and Cloud implementations. What does not yet exist is a formal, versioned contract that makes this property hold for an independently developed model such as PISM by construction rather than by convention: a user can move a validated workflow from a workstation to an institutional cluster or a cloud account supported by the user's own allocation or credits without rewriting the scientific application. Conversely, a newly integrated model can reuse authentication, workspaces, experiment history, remote execution, result delivery, and visualization instead of developing parallel infrastructure. The reduction of duplicated delivery software is central to CryoStack's sustainability model.

CryoBench will turn this contract into an acceptance gate. Unit and schema tests will check interfaces; manufactured/analytic and restart tests will check model behavior; seeded DA tests will compare error and spread diagnostics against tolerances; scaling tests will record cost; software bills of materials and vulnerability scans will check deployability; and a complete provenance manifest will be required before release. CryoBench will also qualify cross-backend scientific equivalence: the question is not whether a workflow merely executes on Local, PACE, and AWS, but whether the same experiment remains scientifically equivalent across them. Equivalence will be defined by application-appropriate, versioned tolerance envelopes rather than bitwise reproducibility — field norms, integrated quantities, conserved or diagnostic quantities, ensemble statistics, and DA-specific diagnostics such as error and spread, depending on the model and experiment. This

is deliberately distinct from performance portability: CryoBench will report, but never require, matching runtime or cost across systems, only that a qualified backend produces a scientifically acceptable answer. Dataset adapters will declare source identity, schema, coordinates, units, uncertainty and quality fields, transformations, licenses, products, and validation fixtures. This is transformative for the target community because a model becomes a reliable, launchable service only after its scientific and operational assumptions are explicit.

3 Proposed research

3.1 System Architecture

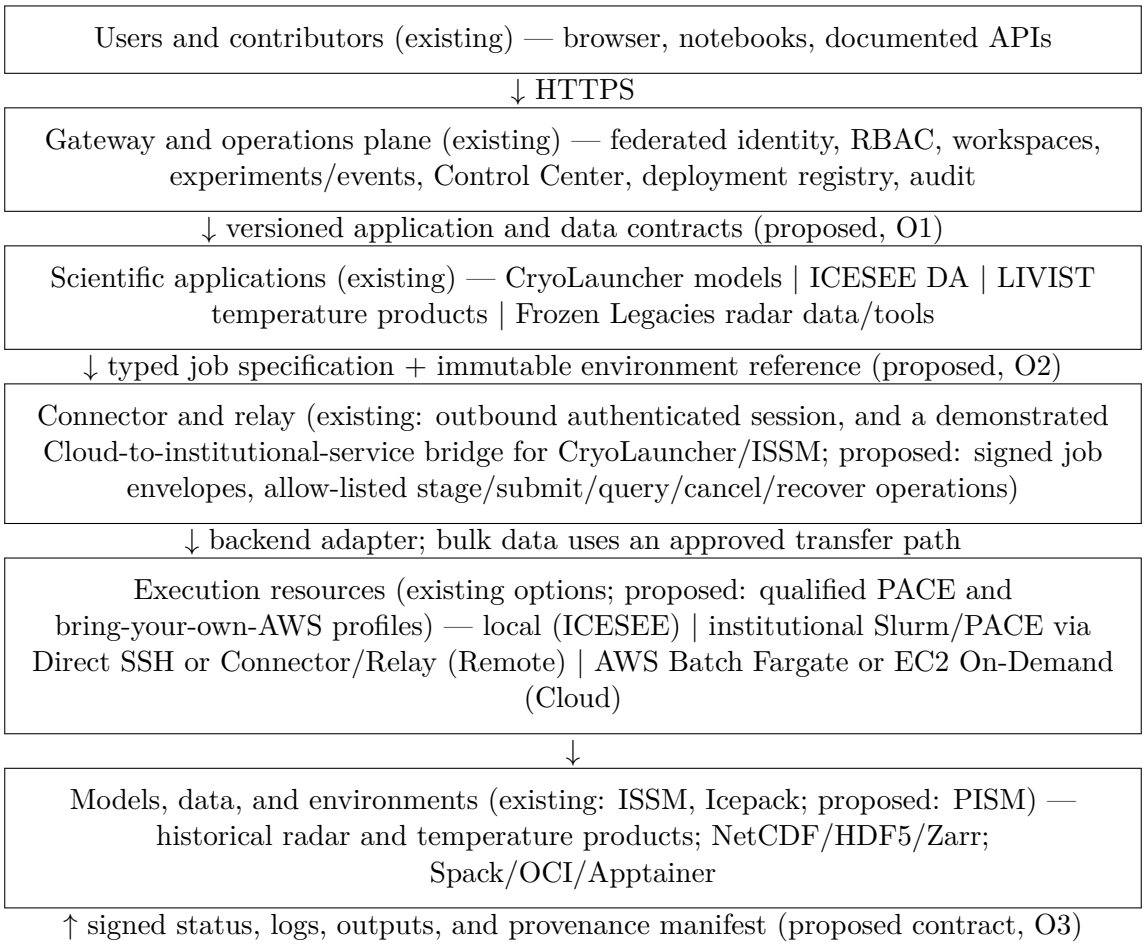


Figure 1: CryoStack's existing foundation and the CSSI advances proposed on top of it. The underlying CryoStack layers shown are present in the current prototype; italicized, proposed annotations identify capabilities that will be generalized, hardened, added, or scientifically qualified through this award. The relay will carry typed control messages and state, not arbitrary shell access or bulk scientific data, and credentials will remain at the narrowest user- or site-controlled layer.

Relative to this existing foundation, the award funds three concentrated advances. O1 replaces ad hoc model/data integration with versioned extension contracts, independently tested by PISM and a second Frozen Legacies collection and gated by CryoBench's scientific and cross-backend equivalence tests. O2 replaces the connector's broad shell-command channel with the typed, allow-listed control plane shown above and qualifies PACE and bring-your-own-AWS execution, including representative parallel workloads where the science requires them. O3 replaces single-node, SQLite-

backed state with a durable, versioned experiment/result/provenance contract, evaluated with real users and sustained through public governance.

The proposed FastAPI service will expose versioned endpoints whose Pydantic/JSON schemas define experiment creation, validation, authorization, submission, cancellation, artifact registration, and state transitions. Its OpenAPI contract will make the control boundary testable; scientific adapters will remain independent of the web framework. Each experiment receives a unique identifier and an immutable configuration snapshot. A job specification references an adapter version, observation-bundle checksum, container digest or Spack lock file, resource request, random seed(s), and requested diagnostics. Status transitions, scheduler job ID, backend, timestamps, exit code, logs, and output locations are recorded. On completion, a JSON manifest binds these fields to code commits, image digests, dependency inventory, and input/output checksums.

3.2 Qualified application, model, and data extension kits (O1)

Task 1.1 – Extension specifications and modular migration. The O1 research question is whether versioned contracts can capture the state, configuration, provenance, and validation needs of models and datasets that differ in interfaces, formats, computational requirements, and scientific use, without erasing their application-specific meaning. Model contracts describe state, forecast, restart, and observation-operator behavior; dataset contracts describe source identity, coordinates, uncertainty, review status, and product lineage. The contribution is a tested method for extending scientific cyberinfrastructure, evaluated by PISM (Task 1.2) and a second historical-radar collection (Task 1.4) rather than by the components that shaped it. During months 1–9, the team will finish separating the current CryoLauncher panels, state objects, execution managers, remote drivers, cloud drivers, and container services from legacy gateway callbacks. Versioned application, model, dataset, and observation-bundle contracts will use typed Python interfaces and JSON Schema, semantic versioning, deprecation rules, reference fixtures, and template repositories. A compatibility matrix will state which application, model/data adapter, environment, and backend combinations are supported. The dependency-aware deployment registry will become the common application packaging and health contract.

Task 1.2 – PISM integration as an independent test of the extension architecture. CryoLauncher, ISSM, Icepack, and ICESEE helped motivate Task 1.1's emerging adapter interfaces, so their fit is not a strong test of generality. PISM did not shape those interfaces: it is an independently developed model with its own state representation, I/O, and restart conventions. Its integration therefore tests whether Task 1.1's versioned contracts generalize to a model the team did not design around, not merely whether one more model can be added. During months 4–18, the team will wrap PISM restart and NetCDF I/O, map selected state/parameter fields, implement observation operators, and support deterministic ensemble-member execution using only the adapter's public interface. The team will classify every integration change by code review: changes confined to the adapter and its declarative/configuration layer count toward a target of at least 90%, and any remaining change to shared CryoStack code must enter through a documented extension interface rather than a model-specific branch or special case, as a direct, auditable measure of whether the extension kit reduces integration burden. The proof of concept will reproduce a documented PISM regional or idealized case and a synthetic-truth assimilation. A production release requires conservation/restart checks, expected error reduction without ensemble collapse, reproducibility within stated tolerances, and successful, scientifically equivalent runs on two qualified execution backends. The team will seek design review from PISM developers or experienced users; no changes to PISM itself are required for core delivery.

Task 1.3 – CryoBench. During months 6–24, the team will implement a tiered validation pipeline: fast checks on every pull request; model-specific CPU tests on merge; scheduled MPI/container tests; and release-candidate tests on supported backends. The release-candidate tier is where cross-backend scientific equivalence is enforced: a candidate adapter/backend combination must reproduce its reference tolerances on every qualified backend it claims to support, not merely execute on it. Results, reference data, tolerances, runtime, and software versions will be published. A failed required tier blocks promotion to the CryoStack production registry. By month 30, every supported adapter will have an identified maintainer, test owner, compatibility statement, and documented scientific limitations. CryoBench criteria are application-appropriate. Computational adapters are judged on numerical consistency, scientific equivalence across backends, configuration and environment reproducibility, and recorded performance and resource use; data adapters are judged on identifiers, coordinates and units, checksums, provenance completeness, and review status rather than on simulation benchmarks. Computing cost is recorded as a run attribute, not used as a qualification criterion.

Task 1.4 – Frozen Legacies extension and data qualification: an independent test of dataset extensibility. Where Task 1.2 tests whether the model-adapter contract generalizes to an independently developed model, this task tests whether the parallel dataset-adapter contract generalizes beyond the single LYRA adapter it was designed around. During months 7–24, the project will turn the existing manifest, adapter, catalog, GeoJSON, map, and documentation implementation into a documented dataset SDK. One additional historical radar collection will be ingested. Qualification will validate identifiers, coordinates, units, coverage, checksums, licenses, source and transformation provenance, quality fields, downloadable products, and geospatial continuity. At least one ASTRA, ARIES, TERRA, or URSA processing path will be wrapped as a reproducible noninteractive task with captured parameters and QC outputs, while retaining interactive tools for expert review.

3.3 Reproducible, secure, and resource-aware execution (O2)

The O2 research question is whether a declared scientific experiment can keep its scientific meaning and provenance when security, isolation, and resource policies are enforced by systems as different as an institutional cluster and a cloud account. The proposed work is not to build an HPC connector; a connector already runs on a user workstation, login/edge host, or institution-approved service near the compute resource, and already establishes an outbound authenticated session to the relay, avoiding any requirement for inbound access through an institutional firewall. The same connector/relay infrastructure already also lets a Cloud workload reach a private institutional service when required – the demonstrated case is CryoLauncher's ISSM workflow on AWS reaching the Georgia Tech institutional MATLAB license – which validates one institutional Cloud/Connector configuration rather than a general private-service bridging capability. The proposed work is to transform that existing proof-of-concept connector into a constrained, auditable control plane suitable for bridging a public gateway to protected institutional resources: the relay will queue signed, typed job envelopes and return state events instead of exposing an interactive shell, and large inputs and outputs will use a configurable approved transfer path rather than passing through the control relay.

Task 2.1 – FastAPI control plane, threat model, and connector hardening. The current desktop connector proves that a browser workflow can reach user-authorized HPC resources, but its general shell-command channel is too broad for a production service. In months 1–9, the project will document assets, trust boundaries, actors, misuse cases, and incident procedures. The connector protocol will be replaced with allow-listed, schema-validated operations (stage declared files, submit a generated batch script, query/cancel an owned job, fetch declared outputs). Commands will not

accept arbitrary shell strings. Paths will be canonicalized into per-experiment directories; resource limits and input sizes will be enforced; logs will redact secrets; and dependencies/images will be pinned and scanned.

Private keys and passwords will never be stored in the public gateway database. For campus resources, credentials will remain in the user's local connector or an institution-approved credential service. Connector sessions will be bound to authenticated users, registered resource profiles, explicit capabilities, expiry, and revocation; command envelopes will include ownership, nonce, and replay checks. The gateway already enforces login-protected routes, gateway-only notebook access, per-user grants, and an allow-listed notebook environment, covered by automated regression tests and a rehearsed restore. The project will extend these tests to session rotation, CSRF protection, rate limits, patching, vulnerability scanning, and incident response, and will isolate notebook kernels in two layers: a per-user sandbox that exposes only the user's own workspace (implemented and tested, not yet enabled in production) and a separate service account for the notebook servers. An acceptance test will verify that a kernel cannot reach gateway credentials, other users' workspaces, or operator keys. An independent Georgia Tech security review will be requested by month 18.

Task 2.2 – Reproducible environments. The team will consolidate the existing ICESEE-Containers and ICESEE-Spack recipes into supported profiles. Each release will publish source provenance, lock files, OCI/Apptainer image digests, software bills of materials, license notices, and smoke-test results. The Year-1 release will reconcile the present BSD/MIT metadata discrepancy and document third-party license constraints, including MATLAB-dependent ISSM paths. Open PISM/Icepack paths will provide the default no-proprietary-license tutorial experience.

Task 2.3 – Backend qualification. Qualification deliberately spans two different infrastructure regimes: PACE, an institutionally managed HPC environment with scheduler, identity, policy, and protected-resource constraints; and AWS, a researcher- or project-controlled cloud environment with its own IAM, Batch, object storage, quota, cost, and lifecycle concerns. Both will be qualified through the same scientific experiment and CryoBench acceptance framework, without requiring identical implementations: the scientific question is whether a qualified workflow retains valid, reproducible scientific meaning across these heterogeneous environments, not whether the environments themselves behave alike. PACE remains the primary development and science-validation resource. AWS is used for portability and elastic demonstration, with budget and quota controls. CryoStack does not require AWS: Slurm/PACE execution remains fully supported, and AWS bills cloud computing to the account that runs it – by default the researcher's own account, and the project account only for the bounded qualification and access activities budgeted here. Building on the existing bring-your-own-AWS path – which already connects a researcher's own account through connection-scoped CloudFormation onboarding, so independently created connections do not depend on one globally fixed execution role, and on which CryoLauncher's ISSM and Icepack workflows have already completed end-to-end execution on both AWS Batch Fargate and EC2 On-Demand, including MPI-parallel ISSM solver execution, postprocessing, result retrieval, and visualization, while ICESEE's own Cloud path has separately completed a single-process (NP=1) Lorenz-96 run on Fargate – the project will qualify this path for sustained scientific use: per-user object isolation in place of a single account-wide bucket, allow-listed Batch job definitions in place of a caller-supplied one, budgets, quotas, lifecycle cleanup, failure recovery, cost attribution, and least-privilege credential handling, tested against a real AWS account rather than fakes alone. These verified results are useful preliminary evidence precisely because they expose the boundary between architectural capability and scientific qualification: CryoLauncher's ISSM and Icepack workflows and ICESEE's Lorenz-96 workflow already execute end to end on AWS, but qualifica-

tion, not new execution architecture, is what makes a configuration trustworthy for production use. CryoLauncher's ISSM workflow already demonstrates single-node MPI-parallel execution on Cloud; where a scientific use case requires it, this capability will be scientifically qualified through CryoBench and extended to multi-node execution and to ICESEE's own multi-process ensemble workflows on Cloud. The project does not commit to a general-purpose cloud-MPI platform independent of the scientific use cases it serves. A backend is "qualified" only if it passes staging, submission, monitoring/cancellation, output recovery, provenance, failure-recovery, and the cross-backend scientific-equivalence tests defined in CryoBench (Task 1.3). The project will not claim identical performance across systems; resource profiles and measured cost/time will guide users.

Task 2.4 – Policy-controlled computing allocations. CryoStack will support two independent AWS execution models – a researcher's own account, billed by AWS to that account, and CryoStack-managed computing funded by the project – while institutional HPC execution remains independent of both. The research question is whether shared execution infrastructure can enforce per-user and per-pool resource policies, and attribute every run to the correct account and funding source, when the authoritative cost record (AWS billing) arrives hours after the run. During months 13–30, the project will develop an allocation service administered from the Control Center: funded computing pools; introductory allocations that can be enabled, disabled, and sized; allocations to individuals or approved groups that can be raised, reduced, or revoked; eligible applications, workflows, and resource profiles; job-duration and per-user concurrency limits; and an append-only record of every allocation change and charge. Allocations are internal accounting units, not AWS promotional credits. Before submission a run reserves its estimated maximum cost (resource profile and duration limit plus storage and transfer allowances); the reservation is reconciled against measured runtime and later billing records, with explicit treatment of failed and interrupted jobs, cleanup, and estimation error. New submissions stop when an allocation or pool cannot cover a reservation; billing alerts remain a second line of defense, not a limit. The current code uses the host's own AWS credentials when a user has no connected account; the service will require an explicit execution mode and refuse workload submission under host credentials except for authorized administrators. Validation will test account attribution, refusal of unauthorized and over-limit submissions, isolation between pools, recovery after interrupted jobs, auditability of allocation changes, and estimated versus actual cost; dollar-level limits will be claimed only to the precision these tests support. Distributing grant-funded computing through the service requires Georgia Tech approval.

3.4 Reproducible experiment services and community access (O3)

The O3 research question is whether persistent, versioned experiment records and application-specific evaluation make results from four different kinds of cryosphere applications reproducible and usable by researchers outside the development team.

Task 3.1 – Durable identity, experiment, and result/provenance services. The current password, GitHub, and ORCID identities, RBAC, saved configurations, workspaces, experiments, events, and Control Center demonstrate the service model in SQLite. Moving multi-user production state to a transactional shared database with versioned migrations, encrypted backups, retention, and export, and replacing the FastAPI relay's process-local connector registry and pending futures with a durable task/session store, is necessary enabling infrastructure, not itself the contribution of this task. The contribution is a common, versioned experiment/result contract that binds, for every qualified application and backend, the scientific/application configuration, execution metadata, backend and environment identity, scientific outputs, application-specific diagnostics, provenance, software/environment versions, and citation/export information. ISSM and Icepack already export

a structured result package; ICESEE does not, and its data-assimilation diagnostics – ensemble error and spread, innovation statistics, and other analysis/assimilation diagnostics appropriate to the selected filter (for example, analysis increments or rank histograms) – are the immediate concrete case this contract must generalize to, since a data-assimilation experiment's scientific content lives in exactly these diagnostics, not only in a final field. Users will be able to clone a prior configuration, launch it on another qualified backend, and export a citation-ready run bundle built on this contract. A documented observation-bundle schema will connect a Frozen Legacies or LIVIST product to CryoLauncher and ICESEE while preserving data license, uncertainty, QC, transformations, and source provenance. The Control Center will gain resource registration, policy/audit search, live service indicators, and privacy-preserving metric export.

Task 3.2 – Usability and acceptance. The team will conduct two formative evaluations (end of Years 1 and 2) and one summative evaluation in Year 3. Tasks are matched to each application: configure and run a supported model through CryoLauncher and retrieve its results; execute a supported ICESEE assimilation example and interpret its error and spread diagnostics; locate a historical radar dataset in Frozen Legacies and interpret its provenance and review status; and explore a LIVIST englacial-temperature product and identify its inputs and scientific context. Shared tasks are finding a tutorial and reproducing a shared run. Measures include completion rate, time to first result, correctness of a short interpretation item for each scientific task, critical incidents, System Usability Scale (SUS), and qualitative barriers. Studies with human participants will begin only after a Georgia Tech IRB determination. Targets are at least 80% completion of the core workflow and median SUS at least 75 by the Year-3 evaluation. Accessibility review will cover keyboard navigation, focus order, contrast, labels, error messages, captions, and screen-reader checks, aiming for WCAG 2.2 AA conformance on core workflows.

Task 3.3 – Community and governance. One hands-on workshop will be offered each year, complemented by quarterly virtual office hours in Years 2 and 3. Materials will include a laptop-scale path and an HPC path. Participant support will prioritize students, early-career researchers, minority-serving institutions, EPSCoR jurisdictions, and institutions without local HPC staff. A public contribution guide, roadmap, code of conduct, support policy, security-reporting channel, adapter proposal template, and lightweight governance document will be released in Year 1. External contributors will receive review pairing and attribution; accepted adapters must name a maintainer and satisfy CryoBench.

Computing-access pilot. As part of training and evaluation, the project proposes a bounded pilot of project-funded AWS computing for eligible CryoLauncher and ICESEE workflows; Frozen Legacies and LIVIST require no computing credits. Up to 50 approved participants would receive a small introductory allocation through the Task 2.4 service (planning value $25 each, $1,250 or about $1,560 with 25% contingency, subject to institutional approval); larger allocations for approved scientific workflows or training events will be estimated before they are offered. Illustrative estimates from current AWS prices put a small default run at cents, but no measured AWS benchmark yet supports precise per-run costs. The pilot evaluates whether controlled introductory allocations reduce onboarding barriers, measured by first-run and workflow completion and cost predictability, and is separate from CryoBench. Afterwards participants continue on their own AWS accounts, PACE, or other institutional or allocated HPC resources; external credit programs such as CloudBank will be explored but are not assumed.

3.5 Engineering, Release, and Acceptance Process

Work will proceed in public issues and milestone boards. Two-week development cycles will end in reviewed pull requests; protected branches will require tests and review. Releases will be semantically versioned, signed where supported, archived with DOI records, and accompanied by migration notes, compatibility matrices, security advisories, and known scientific limitations. A twice-yearly release train balances stability with iteration. Acceptance decisions will combine CI results, science-lead review, security checks, and user-evaluation evidence. Urgent security fixes may use an expedited process with subsequent public documentation.

3.6 Building on Existing, Recognized Capabilities

CryoStack will reuse rather than duplicate mature investments: PISM, ISSM, Icepack, Firedrake, PETSc, MPI, Slurm, Jupyter Book, Voilà, containers, Spack, Georgia Tech PACE, and the existing Georgia Tech virtual-machine deployment. The team will consult Georgia Tech research-computing, security, and research-software specialists and the wider Science Gateways community for usability, security, and sustainability review. CryoStack will link to upstream model documentation and issue trackers and will contribute fixes upstream when appropriate. The adapter SDK will also document how additional models and tools can enter the qualification process. PISM remains the sole new production model, and one additional Frozen Legacies collection is the sole new production dataset integration promised by this Elements project.

3.7 Close Collaboration and Management Plan

PI Brian Kyanjo will direct the project and lead adapter design, ICESEE integration, CryoBench, execution backends, releases, and technical documentation. Co-PI Alexander Robel will lead science requirements, PISM use-case definition, verification criteria, user evaluation, and community engagement. Both will approve production releases. Georgia Tech PACE/security and web-infrastructure personnel will be consulted for deployment and incident response. PISM developers/users, historical-radar data stewards, Science Gateway specialists, and early adopters will be invited to bounded design and acceptance reviews; letters of collaboration will document commitments before submission.

The team will meet weekly, review risks monthly, and conduct quarterly milestone reviews. The PI owns scope, schedule, and technical decisions; the Co-PI has stop authority for scientifically invalid acceptance criteria. Unresolved decisions will be recorded in an architectural decision record and resolved by consensus or, after one week, by the PI with rationale. The public roadmap distinguishes committed award deliverables from aspirational features. The major risks and mitigations are:

- PISM integration complexity: begin with an idealized/regional restartable case, freeze a minimal state/parameter scope, and validate before adding observations or scale.
- Credential or command-injection risk: typed allow-listed operations, user-held credentials, per-run isolation, quotas, logging, scanning, and external review precede public remote execution.
- Backend variability and cloud cost: qualify only PACE and AWS, retain PACE as the reference, and enforce quotas, lifecycle cleanup, and measured cost profiles in CryoStack itself, since AWS budget alerts notify but do not cap spending; pilot allocations are small and approved per workflow.
- Historical-data heterogeneity: use a bounded second collection, require manifest validation and provenance, and retain expert-reviewed QC rather than promising fully automatic interpretation

of every radar format.

- Low adoption: recruit design partners in Year 1, measure task success before scaling outreach, and use PISM's existing support community rather than creating a competing channel.
- Single-maintainer and delivery risk: require reviews, runbooks, release automation, maintainer rotation, and at least two people able to deploy and roll back each production component. Work is ordered by dependency: the security baseline (Task 2.1) and CryoBench's fast tier precede PISM production work, and the deferrable items – a noninteractive radar-processing path, adapter v2, and pilot expansion – are named in advance.

3.8 Integrated measurable outcomes

3.8.1 Deliverables

1. Application/model/data extension kits, templates, documentation, and compatibility policy; production PISM adapter; second Frozen Legacies dataset adapter; completed modular migration of shared frontend/backend code.
2. CryoBench validation service with public tests, reference artifacts, release gates, cross-backend scientific-equivalence and performance records, and scientific limitations.
3. Auth-bound, allow-listed connector/relay and qualified PACE and AWS profiles; a policy-controlled computing-allocation service with reservation, reconciliation, and audit records; reproducible container/Spack releases with SBOMs and license manifests.
4. Versioned experiment and provenance registry; exportable run bundles; observation-bundle schema and one connected Frozen Legacies- or LIVIST-to-model demonstration; expanded Control Center and operations export.
5. Accessible documentation, three workshops, office hours, user studies covering all four applications, a computing-access pilot report (if the pilot is approved), contribution/governance materials, and annual public metric reports.

Detailed yearly targets and collection mechanisms appear in the required Delivery Mechanism and Community Usage Metrics supplement. Headline targets by the end of Year 3 are: five qualified model/application adapters including PISM, two qualified dataset adapters, two qualified execution profiles; at least 100 annual unique authenticated users; at least 1,000 completed scientific runs in Year 3; eight external adopting groups cumulatively; at least 50 annual workshop participants; median SUS at least 75; at least three external code or documentation contributors cumulatively; and, for the PISM production integration, at least 90% of changes confined to the adapter and its declarative/configuration layer, with any remaining shared-code change entering only through a documented extension interface. Metrics are targets for evaluation, not claims of current adoption.

3.8.2 Sustained and Sustainable Impacts

The sustainability strategy has four layers. Technical: automated tests, immutable environments, runbooks, backups, and low-friction releases reduce maintenance cost. Community: adapters have named maintainers, external contributions receive review support, and governance makes decisions transparent. Institutional: Georgia Tech infrastructure provides the reference gateway and HPC profile while portable recipes and user-managed AWS prevent platform lock-in. Scholarly: releases,

run bundles, and documentation receive persistent identifiers and clear citation guidance. During Year 3 the team will publish a five-year operations estimate, identify minimum service and staffing levels, and select among institutional support, future research awards, community allocations, and a later CSSI Transition to Sustainability proposal based on demonstrated adoption. Core software will remain usable locally even if the hosted gateway is unavailable. Software sustainability is distinct from the continuing cost of computing: the award does not fund open-ended cloud use. After the access pilot, computation continues on user-managed AWS accounts or institutional and allocated HPC, while the software, documentation, and data applications remain usable without project-funded credits.

4 Broader impacts

CryoStack lowers barriers to advanced cryosphere computing by allowing a user to learn on a laptop and then execute the same declared workflow on an institutional cluster or supported user-managed cloud deployment. Open tutorials will support graduate courses, workshops, and self-directed learning in DA, uncertainty, reproducibility, and HPC. Participant recruitment and support will deliberately reach students, early-career researchers, minority-serving institutions, EPSCoR jurisdictions, and institutions without research-software staff. Materials will use accessible web practices and captions/alt text, and evaluation will be disaggregated only when privacy and sample size permit responsible interpretation.

The project addresses distinct barriers. Computational access is addressed by execution on institutional HPC and the researcher's own cloud account, supplemented by the bounded access pilot; software complexity by packaged environments and model adapters; data accessibility by Frozen Legacies and LIVIST, which make historical radar observations and englacial-temperature products discoverable with their provenance; reproducibility by versioned experiment records and CryoBench; differences in expertise by tutorials that span modeling, data assimilation, and data exploration; and limited institutional infrastructure by the hosted gateway and laptop-scale tutorials.

The project will also develop the CI workforce. Contributors will learn typed interfaces, numerical testing, secure remote execution, release engineering, software citation, user research, and cross-disciplinary collaboration. Permissive open-source releases and reusable adapter/validation patterns can benefit environmental modeling communities beyond ice sheets. Ultimately, better access to observation-constrained ice-sheet modeling strengthens the research base used to understand sea-level change, a societal hazard affecting coastal communities and infrastructure.

4.1 Alignment with NSF Directorates and Divisions

The primary alignment is CISE/OAC: the project translates working software into a secure, scalable, measurable, and reusable CI service spanning scientific data, software, and heterogeneous computing. The domain alignment is GEO/Office of Polar Programs, because the scientific users and outcomes concern ice-sheet dynamics, observations, and sea-level-relevant cryosphere research. A secondary alignment is GEO/EAR through geophysical inverse problems and Earth-system modeling. The project also responds to CISE interest in software engineering that advances another scientific discipline. Before submission, the team will discuss scope, directorate fit, and the upper-range Elements budget with the cognizant OAC and OPP program officers.

5 Timeline

Period	O1: applications	O2: execution	O3: community platform
Y1	Adapter v1; PISM proof of concept; fast CryoBench tier	Threat model; typed connector; PACE profile; provenance schema; license reconciliation	Design partners; governance kit; workshop; IRB determination; formative study; baseline metrics
Y2	PISM production release; second Frozen collection; observational demonstration; all adapters gated	AWS qualification; allocation service; SBOM/scanning; run export; security review	Workshop; office hours; second study; access pilot; external contribution
Y3	Cross-model and data-to-model experiments; limitations/performance catalog; adapter v2 if needed	Two backend profiles; allocation validation; recovery exercise; cost/operations plan; stable release	Workshop; summative study; maintainer handoff; five-year sustainability and final metrics

Table 1. Integrated work plan. Production promotion requires scientific, engineering, security, and user-acceptance evidence rather than schedule alone.

6 Personnel

PI Brian Kyanjo is a research software developer specializing in ensemble data assimilation, model coupling, MPI/HPC workflows, reproducible environments, and browser-mediated scientific computing. He is the lead developer of ICESEE and the CryoStack/CryoLauncher prototypes. He will direct the project and lead O1, O2, release engineering, and platform integration.

Co-PI Alexander Robel is a Georgia Tech cryosphere scientist whose work combines ice-sheet modeling, theory, uncertainty, and data assimilation. He will lead scientific requirements and acceptance, the PISM use case, user evaluation, and community engagement across O1 and O3. Georgia Tech PACE, security, and web-infrastructure personnel, experienced PISM users, historical-radar data stewards, research-software specialists, and early adopters will participate through bounded design, security, and acceptance reviews documented before submission.

7 Results from prior NSF support

Co-PI Alexander Robel is PI of NSF Award 2235920, CAREER: What's Past is Prologue: Seamless Assimilation of Past Observations into Simulations of Future Ice Sheets, $780,000, January 1, 2024–December 31, 2028 [U.S. National Science Foundation, 2023].

Intellectual Merit: The award develops methods and open-source software for assimilating modern and geological observations into transient ice-sheet simulations. ICESEE implements four ensemble-filter variants, distributed-memory ensemble execution, adaptive state indexing, parallel HDF5/NetCDF I/O, and state and parameter estimation. ISSM and Icepack applications demonstrate interoperability and strong/weak scaling [Kyanjo et al., 2026a]. That award establishes scientific DA methods; this Elements project is distinct in creating, securing, qualifying, delivering, and evaluating reusable infrastructure for multiple applications and computing environments.

Broader Impacts and publication: ICESEE is openly released, has a citable archival version [Kyanjo et al., 2026b], and is distributed through source, container, and Spack pathways. The award has supported training in scientific software and reproducible ice-sheet modeling; its resulting publication is Kyanjo et al. [2026a].

References Cited

References

Choi, Y., Petty, A., Felikson, D., and Poterjoy, J. (2025). Estimation of the state and parameters in an ice-sheet model using an ensemble kalman filter and observing system simulation experiments. EGUsphere, pages 1–25.

Evensen, G. (2003). The ensemble kalman filter: theoretical formulation and practical implementation. Ocean Dynamics, 53:343–367.

Gillet-Chaulet, F. (2020). Assimilation of surface observations in a transient marine ice sheet model using an ensemble kalman filter. The Cryosphere, 14:811–832.

Kyanjo, B. and contributors (2026). Cryostack: Unified cyberinfrastructure for reproducible cryosphere computing across hpc and cloud environments. Georgia Institute of Technology.

Kyanjo, B., Mayo, T. L., and Robel, A. A. (2026a). The ice sheet state and parameter estimator (icesee) library (v1.0.0): Ensemble kalman filtering for ice sheet models. EGUsphere. Preprint.

Kyanjo, B., Robel, A., and Mayo, T. L. (2026b). Icesee (v1.0.0): Ice sheet state and parameter estimator. Zenodo.

Larour, E., Seroussi, H., Morlighem, M., and Rignot, E. (2012). Continental scale, high order, high spatial resolution, ice sheet modeling using the ice sheet system model. Journal of Geophysical Research: Earth Surface, 117:F01022.

PISM Authors (2026). Pism: Parallel ice sheet model. Open-source ice-sheet and glacier modeling framework.

Shapero, D. R., Badgeley, J. A., Hoffman, A. O., and Joughin, I. R. (2021). icepack: a new glacier flow modeling package in python, version 1.0. Geoscientific Model Development, 14:4593–4616.

U.S. National Science Foundation (2023). Award 2235920: Career: What's past is prologue: Seamless assimilation of past observations into simulations of future ice sheets.